

Read the entire manual before starting work.

Failure to comply with formal requirements will result in failing the exercise.

You can use any LLM to prepare a preliminary program skeleton. Treat the generated code only as a starting point—you must understand, modify, and expand it yourself. The final version of the program must be structurally different from what the model returns after a single prompt. The lecturer reserves the right to ask questions about any code fragment during the class.

1. FASTA Format - Prior Knowledge Required

Learn about the FASTA format specifications used in bioinformatics :

https://en.wikipedia.org/wiki/FASTA_format

Pay special attention to the structure of the header line (line 1) and conventions for breaking sequences into fixed-width lines (typically 60 or 80 characters).

2. Basic requirements

Implement a random nucleotide sequence generator in Python using FASTA format. The program must meet the following minimum requirements:

- The program generates a pseudorandom DNA sequence composed of nucleotides {A, C, G, T}. Use the random module from the standard library for generation (do not use external packages at this point).
- The length of the sequence is an input parameter taken from the user (input()). Implement validation: the value must be a positive integer in the range [1, 100 000]. In case of invalid input, the program should prompt for the value again, rather than exit with an exception.
- The user provides a sequence identifier (ID) and its description (description), as specified in the FASTA header. The ID cannot contain whitespace - implement validation. The description is optional (can be an empty string).
- Save the generated sequence to a file named {ID}.fasta. The file must be in the correct FASTA format: the header line starts with the > character and contains the ID and description separated by a space, and the sequence is broken into lines of exactly 80 characters (the last line can be shorter, of course).
- After generating the sequence, the program prints statistics: the percentage of each nucleotide (with precision to two decimal places) and the GC content (GC-content) as a percentage value.
- The challenge: The program inserts the user's name (retrieved via input()) at a random position in the sequence. The letters of the name are not part of the biological sequence—that is, they cannot affect the calculation of statistics or the declared length of the sequence. The name should be visually distinguishable in the output file (e.g., written in lowercase, while the nucleotides remain uppercase).

3. Required code structure

The code must be organized into functions. Below is the minimum set that should appear in the solution. You can add your own functions, but the following must be present and callable.

```
def generate_sequence (length: int) -> str:
    """Returns a random DNA sequence of the specified length."""

def calculate_stats (sequence: str) -> dict :
    """Returns a dictionary of sequence statistics.
    Keys: "A", "C", "G", "T" ( float values , %),
    "GC" ( float value , %)."""
```

```

def insert_name (sequence: str, name: str) -> str:
    """Inserts a name at a random position in the sequence.
    Name written in lowercase letters."""

def format_fasta ( seq_id : str , description : str ,
                    sequence: str, line_width : int = 80) -> str:
    """Returns a formatted FASTA record as a string."""

def validate_positive_int (prompt: str,
                            min_val : int = 1,
                            max_val : int = 100_000) -> int:
    """Gets an integer from the user in a range.
    In case of an error, repeats the question."""

def main ():
    """You know."""

```

The file must end with the structure:

```

if __ name __ == "__ main __":
    main ()

```

4. Interactions

The expected flow of interaction with the program is shown below:

```

$ python s12345_2026.py
Enter sequence length: -5
Error: value must be an integer in the range [1, 100000].
Enter the sequence length: abc
Error: value must be an integer in the range [1, 100000].
Enter sequence length: 40
Enter sequence ID: Seq_001
Enter a description of the sequence: Random test sequence
Enter your name: Catherine

```

Sequence saved to file: Seq_001.fasta

```

Sequence statistics (n=40):
A: 27.50%
C: 22.50%
G: 25.00%
T: 25.00%
GC- content : 47.50%

```

Sample contents of the Seq_001.fasta file:

```

>Seq_001 Random test sequence
ACTGCCTGAAGTCCATGCTAkatarzynaGCATTGACCTAGCTGACCTGA

```

(Note: The above sequence is shorter than 80 characters, so it fits on one line. Longer sequences require line breaks.)

5. Additional functionalities

Include implementations of at least four of the following features in your program. Each selected feature must be implemented as a separate function (or set of functions) and called from main () .

1. Batch mode , i.e. the user specifies the number of sequences to generate (e.g. 5); the program generates them in a loop, each with a unique ID (e.g. Seq_001, Seq_002, etc.) and saves them all to one FASTA file (multi -FASTA).
2. Configurable nucleotide distribution, meaning the user enters the percentage of each nucleotide (e.g., A=30, C=20, G=20, T=30). The program validates that the sum equals 100%.
3. Searching for motifs .. after generating the sequence, the program searches for the motif specified by the user (e.g. "ATG") and lists all the positions of occurrences (indexing from 1, according to the biological convention).
4. Complementary and reverse complementary sequences, meaning the program generates and displays the complementary strand and the reverse complementary strand (remember what that means from lectures?). Both are added as additional records to the FASTA file with appropriate ID designations.
5. *In silico* transcription - the program generates an mRNA sequence (T to U replacement) and saves it as a separate record in a FASTA file.
6. Translation (transcribing to protein) - the program translates the generated DNA sequence into an amino acid sequence using a standard codon table (<https://www.bioinformatics.org/JaMBW/2/3/TranslationTables.html>).
7. Sliding window analysis i.e. the program calculates GC- content in a window with a user-defined width (e.g. 10 nt) moved along the sequence with a step of e.g. 1. The results are saved to a CSV file (columns: start_position , gc_content).
8. content chart - based on data from sliding analysis window program generates a line graph of GC-content along the sequence. The graph is saved to a file.
9. Identification of open reading frames (ORFs) - the program searches for all ORFs (from the ATG codon to the nearest STOP codon: TAA/TAG/TGA) with the minimum length specified by the user and prints their positions and lengths.
10. Support for IUPAC sequences (<https://www.bioinformatics.org/sms/iupac.html>) - in addition to standard nucleotides, the program supports ambiguous characters (R, Y, S, W, K, M, B, D, H, V, N). When generating a random sequence, the user can specify the probability of an ambiguous character.
11. Validator allows you to load a user-supplied FASTA file and checks its format for correctness: header presence, allowed characters in the sequence, and line width. It also issues a report of any errors.

Tip: Choose features whose implementation you can explain yourself. The lecturer may ask you to discuss the logic behind any code fragment .

6. Formal requirements and submission of work

6.1 Solution file

- The solution is a single .py file (not .ipynb - working in Jupyter notebooks is not allowed).
- File name: s{ album_number }_2026.py (e.g. s12345_2026.py). Submitting a file with a different name is treated as no solution.
- The file must be able to be run without modification using the command: python s12345_2026.py in an environment with Python 3.10+.
- Code must include comments explaining the functionality of individual code blocks. At least one docstring per function is expected, as well as comments for unusual solutions.
- At the beginning of the file, place a header in the comment: album number, date, short description of the program.

6.2 GitHub Repository

- Create a separate repository on GitHub and place the solution file there.

- Share the repository with YOUR tutor of today's exercises - GitHub username: Adv20202 (Adam Kuzdraliński), tutorPJATK (Anna Wąsowska) or araczynska-pjatk (Agata Raczyńska)
- main /master branch at the time of checkout is evaluated. If you're working on branches, merge them before releasing.
- Commit history does not affect the assessment, but its complete absence (a single-commit repository with a full solution) may result in additional verbal verification.

6.3 Deadline

7 calendar days from the day on which the given exercises take place on-site.

A final note: a solution generated entirely by LLM without modifications does not meet the requirements of this exercise. This exercise requires, among other things, decomposing the problem into functions, validating input data, breaking lines in FASTA, and implementing additional functionalities.